
Loop Unrolling in CUDA C

A technical reference on `#pragma unroll`

What it does, why it matters, when to use it, and when to avoid it

Contents

1	Background: What a Compiler Actually Does With a Loop	2
2	What <code>#pragma unroll</code> Does	3
3	Why Loop Overhead Is Especially Expensive on a GPU	3
4	The Register Allocation Effect	4
5	A Concrete Example: Tracing the Kernel	6
6	The Downside: Register Pressure and Occupancy	8
7	Occupancy and the Latency Hiding Tradeoff	9
8	When the Compiler Ignores the Pragma	10
9	Relationship to <code>__forceinline__</code>	10
10	Quick Reference: Decision Guide	11
11	Summary	11

1. Background: What a Compiler Actually Does With a Loop

To understand loop unrolling you first need a concrete picture of what a compiler does with a loop before any optimisation is applied. This section builds that picture from first principles.

Source code is not what the hardware executes

When you write C or CUDA C, you are writing in a high-level language. Before your code runs on any hardware, the compiler (`nvcc` for CUDA) translates it into **machine instructions** — the actual binary operations that the GPU’s execution units understand. This translation process involves many passes, each one transforming the code into a progressively lower-level representation.

The key insight is: **the source code and the machine code do not have a one-to-one correspondence**. A single line of C may expand to many machine instructions, and the compiler has enormous freedom in how it performs that expansion.

What a for-loop looks like in machine instructions

Consider a simple loop:

```
1 float sum = 0.0f;
2 for (int t = 0; t < 6; ++t) {
3     sum += a[t];
4 }
```

Listing 1. A simple accumulation loop as you write it in C

Without any optimisation, the compiler emits a sequence of machine instructions that looks roughly like this (written in pseudocode for clarity):

```
t      = 0                ; initialise counter
sum    = 0.0

LOOP_TOP:
  if (t >= 6) goto LOOP_END    ; branch: have we finished?
  tmp   = load a[t]           ; memory load
  sum   = sum + tmp           ; floating-point add
  t     = t + 1               ; increment counter
  goto  LOOP_TOP              ; unconditional jump back

LOOP_END:
  ; sum now holds the result
```

Listing 2. Approximate machine instruction sequence for the rolled loop

Count the instructions per iteration of the loop body: one branch check, one load, one add, one increment, one jump. Only **one** of those five instructions is the actual computation you care about (the add). The other four are *loop overhead* — bookkeeping required to manage the loop itself, not to do the work.

For a 6-iteration loop, that is $6 \times 4 = 24$ overhead instructions to carry out 6 useful additions.

2. What #pragma unroll Does

#pragma unroll is a **compiler directive** — an instruction to the compiler itself, not to the hardware. It is not executable code. It tells `nvcc`: “the loop immediately below this line has a fixed, known iteration count; please eliminate the loop structure entirely and replace it with explicit, sequential copies of the loop body.”

The compiler transforms the source loop:

```
1 #pragma unroll
2 for (int t = 0; t < 6; ++t) {
3     sum += a[t];
4 }
```

Listing 3. Source code with `#pragma unroll`

into machine code *as if* you had written:

```
1 sum += a[0];
2 sum += a[1];
3 sum += a[2];
4 sum += a[3];
5 sum += a[4];
6 sum += a[5];
```

Listing 4. What the compiler effectively emits after unrolling

There is no counter `t`, no branch, no jump. Just six straight-line add instructions. The loop overhead has been completely eliminated.

Nothing about `#pragma unroll` changes what the code computes. The numerical result is identical. It only changes the *machine instructions* emitted to produce that result. This is purely a performance optimisation.

Partial unrolling

You can also supply an explicit unroll factor:

```
1 #pragma unroll 2
2 for (int t = 0; t < 6; ++t) {
3     sum += a[t];
4 }
```

Listing 5. Partial unroll: unroll by a factor of 2

This unrolls the loop two iterations at a time, producing three groups of two additions, each group still guarded by a single branch. It is a compromise between full unrolling and no unrolling, useful when full unrolling is too aggressive (see Section 6).

3. Why Loop Overhead Is Especially Expensive on a GPU

Loop overhead exists on CPUs too, but modern CPUs have sophisticated mechanisms to mitigate it: branch predictors that anticipate the loop back-edge, out-of-order execution that runs the increment and branch in parallel with the useful work, and macro-op fusion

that collapses multiple instructions into one. GPUs have none of these. Understanding why requires a brief detour into how a GPU executes instructions.

The GPU instruction pipeline is simple by design

A GPU's Streaming Multiprocessor (SM) executes instructions in a strict, in-order fashion. It does not speculate ahead, does not reorder instructions, and does not predict branches. Every instruction that appears in the instruction stream costs at least one instruction-issue slot.

The reason GPUs are fast despite this simplicity is that they hide latency through **warp switching**: when one warp is stalled waiting for a memory fetch, the SM switches to another resident warp and executes its instructions instead. This keeps the execution units busy without needing to predict the future. But this strategy only works if there are enough resident warps — and the number of resident warps is limited by, among other things, how many registers each thread uses (more on this in Section 4).

Branches cause warp divergence

Recall that a warp is 32 threads executing the same instruction in lock-step. When those 32 threads hit a branch (`if`, `goto`, `for` condition check) and some threads take the branch while others do not, the warp **diverges**. The hardware must then execute both paths sequentially, masking off the threads that should not execute each path. Half the execution capacity is wasted for the duration of the divergence.

In a short, fixed-count loop like:

```
1  for (int t = 0; t < 6; ++t) { ... }
```

all 32 threads in a warp will always follow the same path (they all iterate 6 times), so there is no *divergence* here. But there is still overhead: the branch instruction itself must be issued, evaluated, and the jump executed on every iteration, consuming instruction-issue bandwidth that could otherwise be spent on useful floating-point operations.

Instruction-issue bandwidth is a finite resource

An SM can issue a fixed number of instructions per clock cycle. On a modern NVIDIA GPU (Ampere architecture), an SM can issue up to 4 instructions per clock cycle across its warp schedulers. Every loop overhead instruction (counter increment, branch check, back-jump) consumes one of those issue slots. Eliminating overhead instructions via unrolling directly frees those slots for floating-point arithmetic, improving **arithmetic throughput**.

For a kernel like `cov_reduce_r3_sym6_warp` that is compute-bound in the inner accumulation loop, this matters. If roughly half your issued instructions are loop bookkeeping and half are multiply-adds, you are running at half your theoretical peak arithmetic throughput.

4. The Register Allocation Effect

This is the subtler and arguably more important effect of `#pragma unroll`, and the one that is hardest to internalise without thinking carefully about what a compiler can and cannot see.

What registers are

Registers are the fastest storage on the GPU — approximately 1 clock cycle access latency, compared to ~ 400 – 800 cycles for global memory. Each SM has a fixed pool of 65,536 32-bit registers, shared among all threads currently resident on that SM. A thread with more register usage leaves less room for other threads and reduces occupancy (the fraction of the SM’s maximum thread capacity that is in use).

When the compiler cannot fit all of a thread’s live variables into registers, it **spills** some of them to **local memory**, which physically resides in global VRAM. A register spill turns a 1-cycle access into a 400–800 cycle access. This is catastrophic for performance if it happens inside an inner loop.

What the compiler can see in a rolled loop

In a rolled loop, the compiler sees only one iteration at a time when it performs its analysis. It processes the loop body as if it could execute an arbitrary number of times with any value of `t`. Consider:

```

1  float acc[6];
2  for (int t = 0; t < 6; ++t) acc[t] = 0.0f;
3
4  for (int i = start + tid; i < stop; i += T) {
5      /* compute d0, d1, d2, wd0, wd1, wd2 ... */
6      acc[0] += wd0 * d0;
7      acc[1] += wd0 * d1;
8      /* ... */
9      acc[5] += wd2 * d2;
10 }
```

Listing 6. A rolled loop over the accumulator array

In this code `acc` is an array. The compiler may or may not be able to prove that `acc[0]` through `acc[5]` are all independent and always accessed at fixed offsets. If it cannot prove this — and with a rolled initialisation loop it has less information — it may allocate `acc` as an array in **local memory** (VRAM), not as 6 independent registers. Every `acc[t] +=` then becomes:

1. A global memory load of `acc[t]` (~ 500 cycle latency)
2. A floating-point multiply-add (1 cycle)
3. A global memory store of the result back to `acc[t]` (~ 500 cycle latency)

The 1-cycle useful operation is buried under 1000 cycles of memory traffic.

What the compiler can see after unrolling

After unrolling, the compiler no longer sees an array accessed via a variable index. It sees six independent scalar variables:

```

1  /* What the compiler effectively sees after unrolling both loops */
2  float acc0 = 0.0f;
3  float acc1 = 0.0f;
4  float acc2 = 0.0f;
5  float acc3 = 0.0f;
6  float acc4 = 0.0f;
```

```

7  float acc5 = 0.0f;
8
9  for (int i = start + tid; i < stop; i += T) {
10     /* ... */
11     acc0 += wd0 * d0;
12     acc1 += wd0 * d1;
13     acc2 += wd1 * d1;
14     acc3 += wd0 * d2;
15     acc4 += wd1 * d2;
16     acc5 += wd2 * d2;
17 }

```

Listing 7. After unrolling, the compiler sees 6 independent scalars

Now the compiler can trivially prove that `acc0` through `acc5` are six independent scalars with no aliasing between them. It allocates each to a dedicated register. Every `acc_k +=` is a single fused multiply-add (FMA) instruction operating entirely on registers — no memory traffic whatsoever during the inner loop.

The branch elimination is a small win. The large win is that unrolling convinces the compiler to keep all accumulator variables live in registers throughout the entire inner loop, turning what could be thousands of memory round-trips (one per data point per accumulator) into a single set of register-resident running sums. This is the true reason `#pragma unroll` appears on the initialisation loop — it is not about the initialisation itself, it is about establishing the accumulators as independent register-resident scalars before the expensive loop begins.

Fused Multiply-Add (FMA) instructions

When the compiler sees the pattern `acc += a * b`, it can often emit a single **FMA** instruction rather than a separate multiply and add. An FMA computes $a \times b + c$ in one clock cycle with a single rounding step, which is both faster and more numerically accurate than two separate operations. Unrolling helps here too: by exposing all six accumulation expressions simultaneously, the compiler has maximum freedom to schedule FMA instructions and interleave them to keep the GPU's floating-point execution units continuously busy.

5. A Concrete Example: Tracing the Kernel

Let us trace through the exact usage in `cov_reduce_r3_sym6_warp` step by step to make the above concrete.

Step 1: Initialisation loop

```

1  float acc[6];
2  #pragma unroll
3  for (int t = 0; t < 6; ++t) acc[t] = 0.0f;

```

Listing 8. Unrolled initialisation of the accumulator array

Without unroll: The compiler sees `acc` as an array, potentially allocates it in local memory, and emits a rolled loop with a counter, branch, and back-jump.

With unroll: The compiler emits six sequential register assignments and knows from this point forward that `acc[0]` through `acc[5]` are six independent register-resident scalars.

Step 2: The inner accumulation loop

```

1  for (int i = start + tid; i < stop; i += T) {
2      float wi = 1.0f / w[i];
3
4      float d0 = D[i*3 + 0];
5      float d1 = D[i*3 + 1];
6      float d2 = D[i*3 + 2];
7
8      float wd0 = wi * d0;
9      float wd1 = wi * d1;
10     float wd2 = wi * d2;
11
12     acc[0] += wd0 * d0;
13     acc[1] += wd0 * d1;
14     acc[2] += wd1 * d1;
15     acc[3] += wd0 * d2;
16     acc[4] += wd1 * d2;
17     acc[5] += wd2 * d2;
18 }

```

Listing 9. Inner loop: this is where the overwhelming majority of time is spent

This loop does not have a `#pragma unroll` because its trip count (`stop - start`) is *not* known at compile time — it depends on the data. You cannot unroll a loop whose count is unknown. What the pragma on the initialisation loop has done, however, is ensure that by the time this loop begins, `acc[0]` through `acc[5]` are already established as six register-resident scalars. The six `acc[k] +=` statements inside this loop therefore compile to six FMA instructions operating entirely on registers, with no memory traffic beyond the loads of `w[i]` and `D[i*3+0..2]`.

Step 3: The warp reduction loop

```

1  #pragma unroll
2  for (int t = 0; t < 6; ++t) acc[t] = warp_sum(acc[t]);

```

Listing 10. Unrolled warp reduction: all 6 shuffle sequences visible to the scheduler

After unrolling, the compiler sees 30 shuffle instructions (6 entries $\times$ 5 shuffle steps each) as a straight-line sequence. It can interleave them optimally in the instruction stream, potentially issuing independent shuffles back-to-back without stalls. In the rolled form, the compiler would see only one call to `warp_sum` at a time and have less freedom to schedule across iterations.

Step 4: Writing to shared memory

```

1  if (lane == 0) {
2      #pragma unroll
3      for (int t = 0; t < 6; ++t)
4          sh[t * num_warps + warp] = acc[t];
5  }

```

Listing 11. Unrolled write to shared memory

Six independent stores from six register-resident scalars. Without unrolling, this could degrade to a loop over an array in local memory, adding unnecessary load–store traffic before the data even reaches shared memory.

6. The Downside: Register Pressure and Occupancy

Unrolling is not free. It has one significant cost: it increases the number of variables that must be live simultaneously, which increases **register pressure**.

What register pressure means

Each SM has 65,536 registers shared across all resident threads. If each thread uses R registers, the maximum number of threads that can reside on the SM simultaneously is $\lfloor 65536/R \rfloor$, subject to other limits (shared memory size, max threads per block, etc.). This ratio of actual resident threads to maximum possible is called **occupancy**.

High occupancy is desirable because the SM hides memory latency by switching between warps. If only a few warps are resident, the SM has fewer warps to switch to when one stalls, and it sits idle more often.

How unrolling increases register pressure

In a rolled loop, the compiler can reuse a small set of registers across iterations — the loop variable, a few temporaries, and the accumulator. After unrolling, all N copies of the loop body are live simultaneously in the compiler’s view, and it may need to keep more values in registers at once.

Consider unrolling a loop 32 times. The compiler now sees 32 copies of every temporary variable (`d0`, `d1`, `d2`, `wd0`, `wd1`, `wd2`) in the expanded instruction stream. It may need 32 separate registers for each, multiplying register usage dramatically.

Register spilling: when unrolling backfires

If the unrolled loop requires more registers than are available, the compiler **spills** the excess to local memory (VRAM). A spill inserts a global memory store before the variable is last used and a global memory load when it is needed again. This can be slower than the rolled loop you started with — you have eliminated a branch and gained a 500-cycle memory round-trip.

Excessive unrolling can cause the compiler to spill registers to local memory, which resides in slow global VRAM. A spilled accumulator inside an inner loop is catastrophic: every iteration now incurs hundreds of cycles of memory latency for a variable that should live in a register. This is the primary failure mode of over-aggressive unrolling.

How to detect register spilling

The NVIDIA compiler can report register usage per thread:

```
nvcc --ptxas-options=-v my_kernel.cu
```

Listing 12. Compile-time register usage report

This prints output like:

```
ptxas info: Used 32 registers, 48 bytes lmem, 16 bytes smem
```

The `lmem` figure is **local memory** — any non-zero value means register spilling is occurring. You want this to be zero for performance-critical kernels.

At runtime, NVIDIA Nsight Compute will also flag excessive local memory usage and show you which variables are spilling.

7. Occupancy and the Latency Hiding Tradeoff

The interaction between unrolling, register pressure, and occupancy creates a genuine engineering tradeoff that has no universal answer.

The two competing effects

		More unrolling	Less unrolling
Instruction over-head		Lower (fewer branches, jumps)	Higher
FMA scheduling freedom		Higher (more visible to compiler)	Lower
Registers per thread		Higher	Lower
Occupancy		Lower (fewer resident warps)	Higher
Latency hiding ability		Worse	Better

When does each side win?

Unrolling wins when the kernel is compute-bound. If the GPU's floating-point units are the bottleneck (not memory bandwidth), then eliminating overhead instructions and enabling better FMA scheduling directly improves throughput. A kernel processing huge amounts of data with simple arithmetic (like `cov_reduce_r3_sym6_warp` in its inner loop) is often compute-bound.

Rolling wins when the kernel is memory-bound. If threads spend most of their time waiting for global memory loads, the GPU needs high occupancy to have other warps available to switch to during those stalls. Reducing register pressure to fit more warps on the SM is more valuable than saving a few instructions per iteration.

The correct decision depends on your specific kernel and hardware, and the honest answer is that you should profile both versions with a tool like Nsight Compute rather than guess.

8. When the Compiler Ignores the Pragma

`#pragma unroll` is a *hint*, not a command (unless combined with `__forceinline__` semantics in some contexts). The compiler will ignore it or override it in several situations:

- **Trip count not known at compile time.** If the loop bound is a runtime variable (e.g., `for (int i = 0; i < n; ++i)` where `n` is a function argument), the compiler cannot fully unroll. It may partially unroll if a factor is supplied.
- **Loop body is too large.** If fully unrolling would produce an enormous instruction stream that overflows the instruction cache or causes severe register spilling, the compiler may silently decline.
- **Loop contains function calls.** If the loop body calls a non-inlined function, unrolling is limited by what the compiler can see. This is another reason `__forceinline__` and `#pragma unroll` are natural companions.
- **Loop has complex control flow.** Loops with `break`, `continue`, or internal branches may prevent full unrolling.

The compiler will not warn you if it decides not to honour `#pragma unroll`. If you need to verify that unrolling occurred, inspect the PTX or SASS (assembly) output with `nvcc -ptx` or Nsight Compute's source-correlator view.

9. Relationship to `__forceinline__`

These two directives are conceptually related and often used together.

`__forceinline__` tells the compiler to substitute a function's body directly at its call site, eliminating the function call overhead (saving/restoring registers, instruction pointer jump). It operates at the **function boundary** level.

`#pragma unroll` tells the compiler to eliminate the loop structure within a function body. It operates at the **loop** level.

When `warp_sum` is marked `__forceinline__` and its internal reduction loop (implicitly unrolled by the compiler because the 5 shuffle steps are written out explicitly) is called from an unrolled outer loop, the two effects compound: the function body is stamped in directly, and then all copies of it are visible together, giving the scheduler the widest possible view of the instruction stream to optimise.

In general: **inline first, then unroll**. Unrolling a loop that contains non-inlined function calls does less good because the compiler cannot see inside those calls.

10. Quick Reference: Decision Guide

Situation	Recommendation
Small fixed loop (≤ 8 iterations), accumulators in registers	Use <code>#pragma unroll</code> . Almost always a win.
Small fixed loop, loop body contains many temporaries	Use <code>#pragma unroll</code> and check register count with <code>-ptxas-options=-v</code> . Watch for <code>lmem > 0</code> .
Large fixed loop (32+ iterations)	Try <code>#pragma unroll N</code> with partial factor. Profile both.
Loop bound is a runtime variable	Full unrolling not possible. Partial unroll may help.
Kernel is memory-bound (not compute-bound)	Prioritise occupancy over unrolling. May be better without.
Loop body calls non-inlined functions	Apply <code>__forceinline__</code> to called functions first.
You are unsure	Profile with Nsight Compute. Measure, do not guess.

Unroll small, fixed-count loops over accumulator arrays that live inside or immediately adjacent to a hot inner loop. The combination of branch elimination and register promotion is almost always a net win for loops up to about 8–16 iterations. Beyond that, always verify with a profiler.

11. Summary

`#pragma unroll` is a compiler directive that eliminates loop control overhead by expanding a fixed-count loop into sequential straight-line code at compile time. Its effects on GPU performance are:

- **Eliminates loop overhead instructions** (branch check, counter increment, back-jump) from the instruction stream, freeing instruction-issue bandwidth for useful arithmetic.
- **Promotes array elements to independent register-resident scalars**, preventing the compiler from allocating small fixed arrays in local memory (VRAM) and turning every read/write into a hundreds-of-cycles memory operation.
- **Expands the compiler's visibility** of the instruction stream, enabling better FMA scheduling and interleaving of independent operations.
- **Increases register pressure** as a side effect, potentially reducing occupancy. For small loops this cost is negligible; for large loops it can outweigh the benefits.
- **Has no effect on numerical results.** It is a pure performance transformation.

In the context of `cov_reduce_r3_sym6_warp`, the pragma on the 6-element initialisation and reduction loops is not primarily about those loops themselves. Its main purpose is to signal

to the compiler that the 6 accumulator variables are independent scalars that should live in registers for the entire duration of the inner loop over data points — the hot path where the overwhelming majority of the kernel’s execution time is spent.

CUDA Concepts Reference · Applies to NVIDIA CUDA C/C++ compiled with `nvcc` · Behaviour may vary across compute capabilities and compiler versions